

CUSTOMER JOURNEY MAP

Credit Card Fraud Detection System Using Machine Learning

Prepared by: **Bhargavasai** • Internship Project Showcase

SCENARIO

A user or evaluator wants to check whether a specific credit card transaction is legitimate or fraudulent using a real-time ML prediction demo. The web application features a responsive HTML/CSS frontend, a robust Python Flask backend proxying model execution, an advanced machine learning pipeline (XGBoost/Random Forest) integrated via Joblib, and is fully deployed on Render cloud architecture.

DIMENSION	ENTICE Discover the project	ENTER Start prediction	ENGAGE Fill transaction data	ENGAGE Submit to ML API	EXIT Read prediction result	EXTEND Review codebase
Steps	Visits the home page. Reads that the tool provides instant, automated fraud predictions to eliminate manual human error.	Clicks the navigation link to head to the prediction page. Navigates to the input form interface.	Enters the specific transaction details required by the model: Time , V1 to V28 PCA features, and Amount .	Clicks the Predict button. The frontend sends the structured transaction fields to the backend server.	Views the final real-time verdict text: either "Normal Transaction" or "Fraud Transaction Detected."	Opens the GitHub Repository to inspect the source code, trained model files, testing reports, and README.
Interactions	Home page UI, navigation bar, project objective section, and feature cards.	Prediction page routing link, primary Call-to-Action (CTA) button.	Structured input form, numerical text fields for Time/Amount, and 28 feature boxes.	Predict submit button, Flask backend routing, Python request handler, and saved Joblib model object.	Result display card, alert status banner, real-time response interface.	GitHub repository, README .md, project documentation, testing reports, and public Render URL.
Goals	Understand the core purpose and how automation replaces slow, manual fraud checks.	Easily transition from introductory pages to the functional testing interface without friction.	Provide all the necessary mathematical transaction features accurately without validation errors.	Route input parameters to the backend smoothly, trigger the ML model, and process a rapid calculation.	Clearly understand if the tested transaction exhibits high-risk fraudulent patterns or is safe.	Review the end-to-end engineering pipeline including EDA, model selection, evaluation

DIMENSION	ENTICE Discover the project	ENTER Start prediction	ENGAGE Fill transaction data	ENGAGE Submit to ML API	EXIT Read prediction result	EXTEND Review codebase
						metrics, and cloud deployment.

POSITIVE MOMENTS

- **Instant Automation:** Replaces error-prone manual transaction analysis with real-time predictions.
- **Streamlined Flow:** Seamless data passage between HTML/CSS frontend and Python Flask backend.
- **Robust Engineering:** Full pipeline including handling missing values, duplicate removal, and `StandardScaler` normalization.

NEGATIVE MOMENTS

- **Input Complexity:** Manually typing 28 abstract PCA components (V1-V28) can feel overwhelming for non-technical users.
- **Instance Latency:** Free cloud hosting tier (Render) introduces potential spin-up delays during initial requests.

AREAS OF OPPORTUNITY

- **Explainable AI (XAI):** Embed SHAP or LIME visualizations inside the Flask UI to detail **why** a transaction was flagged.
- **Batch CSV Processing:** Add a file upload widget to let evaluators upload a batch of records rather than inputting single items.

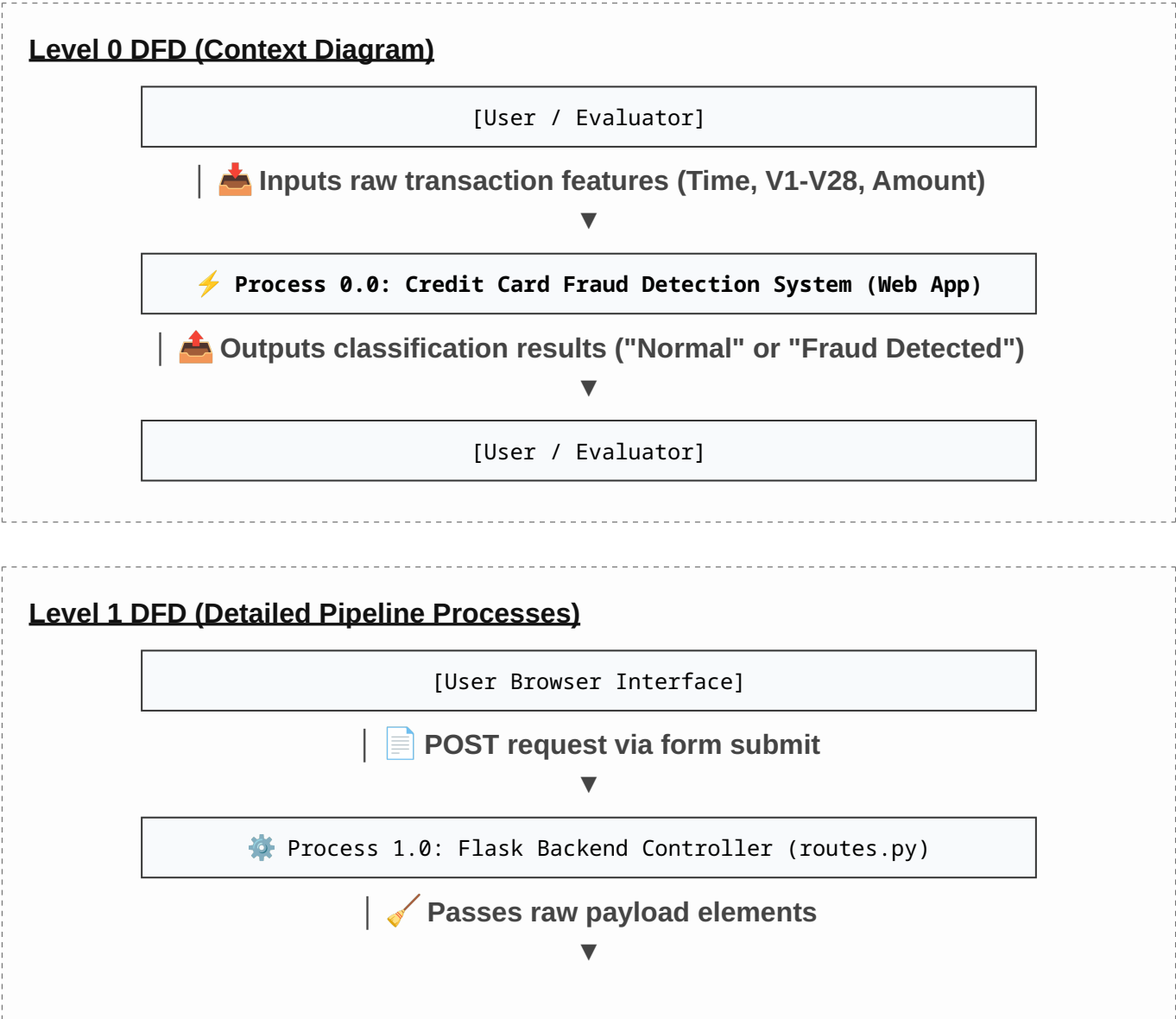
PROJECT DESIGN PHASE-II

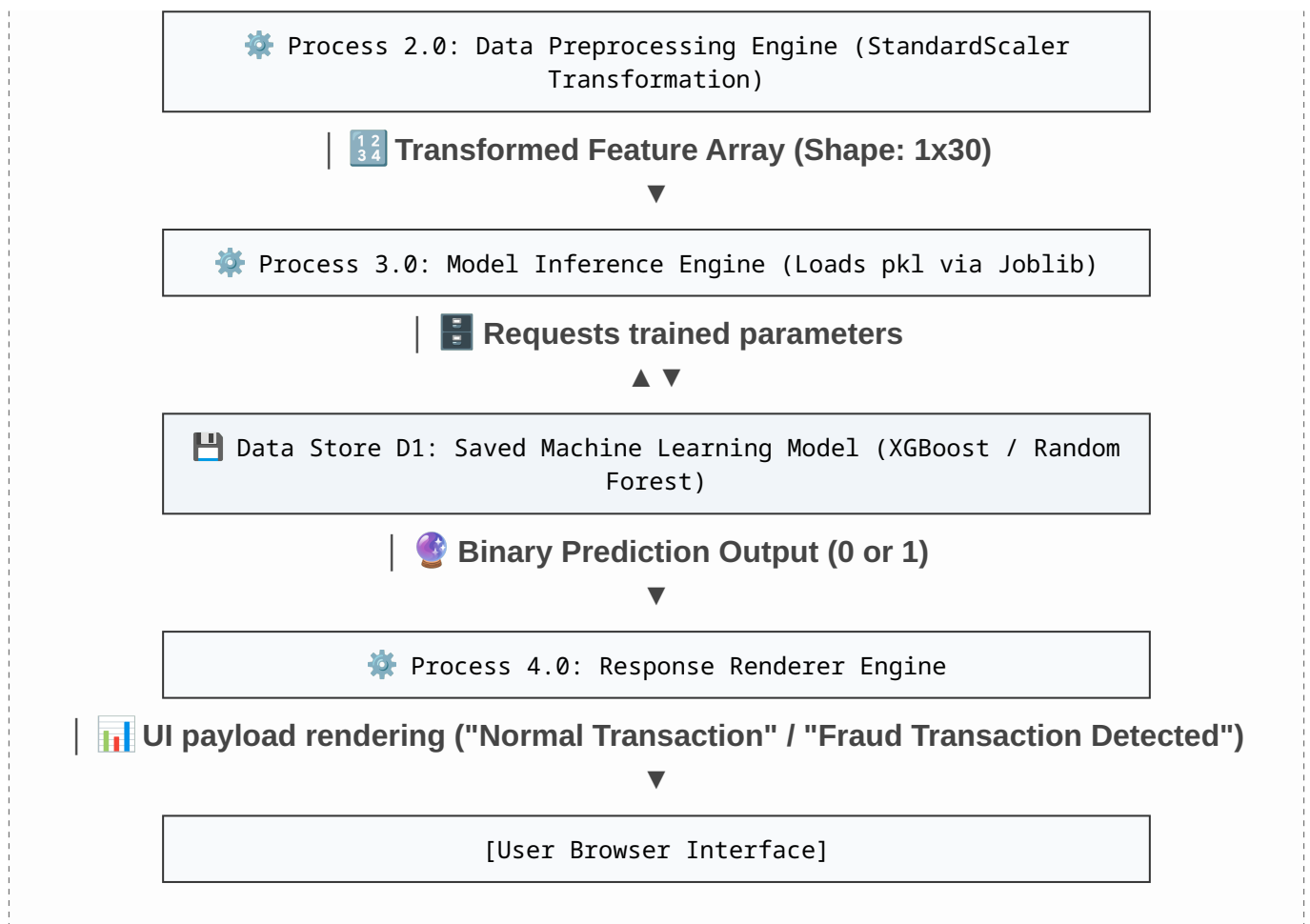
Data Flow Diagram & User Stories

Date	6 july 2026
Team ID	XXXXXX
Project Name	Credit Card Fraud Detection System using Machine Learning
Maximum Marks	4 Marks

1. DATA FLOW DIAGRAM (DFD)

Below is the structured functional flow of data through the machine learning pipeline, from the frontend interface to model inference and backend evaluation.





2. CORE AGILE USER STORIES

- **User Story 1:** As a bank compliance evaluator, I want to submit raw numeric PCA features (V1 to V28) alongside transaction amount and time so that I can immediately verify if a transaction shows high-risk fraudulent anomalies without human manual review.
- **User Story 2:** As a project reviewer, I want to access the application via a public Render URL without local environment setups so that I can rapidly audit the runtime behavior and verify live prediction speeds.
- **User Story 3:** As a system developer, I want to inspect the complete version-controlled code, EDA visuals, and evaluation metrics on GitHub so that I can reproduce, scale, or integrate the pipeline into core banking microservices.